7-1 · LangChain 챗봇 기초 · 수업 전 10분

실습점검 · 환경재설정

OpenAI 키 또는 Ollama · 11434

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

왜 점검하나

수업 전에 걸러 둘 것

① 이번 주는 LLM 을 부른다 — OpenAI API 또는 내 PC 의 Ollama

② 코드는 같고 rag_env.py 한 곳만 갈린다(실습 1단계에서 만든다)

③ Ollama 모델은 수 GB — 수업 중에 받으면 그 시간이 통째로 날아간다

1~4단계는 확인과 준비 · 가상환경과 rag_env.py 는 실습 1단계 · 재설정(6단계)은 처음부터 할 때만

흐름

여섯 단계 — 위에서부터 한 번씩

OpenAI 를 고른 사람은 3 · 4단계를 건너뛰고 5단계로 갑니다

1. 6주차 멈춤	2. 경로 정하기	3. 모델 받기	4. 응답 확인	5·6. 순서 · 재설정
compose stop 메모리 비우기	OpenAI 키 또는 Ollama	llama3.2 embeddinggemma	11434 api/generate	아직 치지 않음 처음부터 할 때만

1단계

6주차 컨테이너를 멈추세요

1. **cd ~/data-eng/week6 → docker compose stop**
   connect · kafka · mysql · cdc-minio 가 Stopped(실측)
2. **docker ps**
   제목 줄만 남으면 된다 — 이미 멈춰 있으면 stop 은 조용히 지나간다
3. **지우지 않고 멈추기만**
   6주차 산출물 정리 때 다시 켠다

이유

메모리를

Ollama 에

2단계

경로를 정하세요 — 한 주 내내 같은 경로

	OpenAI API	로컬 Ollama
준비	키 1개 · 결제 수단	모델 내려받기 2.0GB + 621MB
비용	쓴 만큼	없음
속도	한 답 몇 초	PC 에 달림 한 문장 약 10~30초
모델	gpt-4o-mini 임베딩 1536차원	llama3.2 embeddinggemma 768차원

임베딩 차원(1536 · 768)이 7-3 컬렉션 설정에 들어간다 — 중간에 바꾸면 7-3 부터 다시 적재

3단계 · Ollama 경로

Ollama 를 도커로 띄우고 모델을 받으세요

1. **docker run -d --name ollama -p 11434:11434 … ollama/ollama**
   이미지는 처음 한 번 3.39GB — 디스크에는 8.47GB(실측)

2. **docker exec ollama ollama pull llama3.2 · embeddinggemma**
   실측 llama3.2 약 3분 · embeddinggemma 약 2분(회선에 따라 다름)

3. **docker exec ollama ollama list**
   llama3.2:latest 2.0 GB · embeddinggemma:latest 621 MB 두 줄

4. **Conflict … "/ollama" is already in use**
   이미 만들어 둔 것 — docker start ollama 만 친다

받는 양

2.0GB

+ 621MB

3단계 · 설치형을 쓰려면

도커 대신 설치형 — 운영체제별

01

Windows (WSL2)

WSL 안에서 리눅스용을 설치

Windows 앱은 안 닿을 수 있음

02

macOS

Apple Silicon 은 앱이 GPU 로 빠름

Intel 맥은 도커 경로 권장

03

리눅스

install.sh 한 줄 · 서비스로 뜸

명령에서 docker exec ollama 를 뺀다

어느 운영체제든 같은 명령이 되도록 이 과정은 도커 경로를 기본으로 씁니다

4단계 · Ollama 경로

11434 포트와 응답을 확인하세요

1. **curl http://localhost:11434**
   Ollama is running(실측)

2. **curl -s .../api/generate -d '{"model":"llama3.2","prompt":"안녕",...}'**
   {"model":"llama3.2", ... "response":"안녕하세요! ... 로 시작하는 JSON

3. **curl: (7) Failed to connect**
   컨테이너가 떠 있지 않은 것 — docker start ollama

4. **한 문장에 몇 분씩 걸리면**
   스레드 수 — num_thread 6 으로 줄여 초당 0.3~0.5 → 4~7 토큰(실측)

처음 부를 때

모델을 올리느라 더 걸린다

5단계 · 순서만

오늘 만드는 것 — 아직 치지 않는다

무엇	어디서 처음 만드나	남는가
~/data-eng/week7 · .venv	실습 1단계	예 — 7-5 까지 그대로
.env · .gitignore (OpenAI 경로만)	실습 1단계	예 — .env 는 Git 에 안 올림
rag_env.py (모델 설정 한 곳)	실습 1단계	예 — 이번 주 전 실습이 씀
call.py … temp.py (여섯 파일)	ChatModel 실습 2~7단계	예
nohistory.py … chatbot_trim.py (여섯)	챗봇 실습 1~6단계	예

단계마다 cat > 파일.py << 'EOF' … EOF → python 파일.py 순서로 만든다

6단계

환경 재설정 — 오늘 실습 파일만 지운다

1. **cd ~/data-eng/week7 → rm -f call.py ... chatbot_trim.py**
   오늘 만드는 실습 파일 열두 개만 지운다

2. **.venv · .env · rag_env.py 는 남긴다**
   이것까지 지우면 실습 1단계부터 다시

3. **Ollama 모델은 지우지 않는다**
   다시 받으려면 수 분

언제

처음부터

다시 할 때만

7-1 · 수업 전 10분 마무리

다음은 LLM API와 LangChain 아키텍처 (1H)

LLM 을 어떻게 부르는지, LangChain 이 무엇을 대신해 주는지 이번 주 지도를 잡습니다

- ✓ **LLM 의 세 가지 한계와 RAG**
- ✓ **ChatModel · 템플릿 · 파서 · 체인**
- ✓ **메시지 세 종류와 토큰 비용**

막혔다면

그 단계 아래 「막혔어요」 · 「질문」